

발표 자세 코칭 앱 만들기 (Poise 기준)

이 유인물만 보고 따라 하면 브라우저에서 바로 돌아가는 발표 자세 코칭 프로토타입을 만들 수 있음. 레퍼런스는 실제 배포된 앱 **Poise**(<https://poise-deploy.vercel.app/>) 하나임 — 이 앱의 화면 구성과 흐름을 그대로 따라감. 서버 연결, API 키 발급, 배포 같은 건 다 뺐음. HTML 파일 하나 만들어서 크롬으로 열면 끝.

0. 뭘 만드는 건지 먼저 알기

카메라 앞에서 발표 연습을 하면, **연습 중에는** 자세를 실시간으로 인식해서 안 좋은 습관이 나올 때 코칭 문구를 띄워주고, **연습이 끝나면** 녹화본이랑 같이 어떤 습관이 몇 번 나왔는지 리포트로 정리해주는 앱.

헷갈리기 쉬운 부분이라 먼저 정리하면: 이 앱은 "실시간 경고"랑 "끝난 뒤 리포트" 두 가지를 다 함. 실시간 경고만 만들면 Poise 절반밖에 안 만든 거고, 리포트만 만들면 연습 중 피드백이 없는 거임. 이 튜토리얼은 둘 다 순서대로 만듦.

감지할 자세 클래스 (5개)

클래스	의미
정상	문제없는 자세
시선 회피	카메라를 안 보고 다른 곳을 봄
고개 움직임	고개가 과하게 움직이거나 숙여짐
상체 흔들림	상체가 좌우로 흔들림
발화 멈춤	몸 전체가 일정 시간 움직임 없이 멈춰 있음 (2초 이상 정지)

화면 흐름은 6단계로 나뉨 (Poise를 그대로 따라감, 이거 기준으로 UI랑 기능을 같이 설계할 거임)

순서	화면	언제 뜨는지
1	촬영 가이드	앱 처음 켤 때

순서	화면	언제 뜨는지
2	홈	가이드 확인하고 "시작하기" 누른 직후
3	카운트다운	"연습 시작하기" 버튼 누른 직후 (3초)
4	실시간 녹화	카운트다운 끝난 직후 ~ 길게 눌러서 종료할 때까지
5	분석 중	녹화 종료 직후
6	세션 리포트	분석 끝난 직후

(자주 묻는 질문) 왜 6단계씩이나 만들어야 하나요? 그냥 실시간 경고만 있으면 안 되나요? Poise는 딱 실시간 경고만 있는 앱이 아니라 "연습 → 복기" 사이클이 핵심 컨셉임. 리포트 화면이 없으면 연습이 끝나고 뭘 고쳐야 할지 남는 게 없어서, 이 6단계 흐름 자체가 이 프로젝트의 핵심 산출물임. 화면 수가 많아 보여도 6번(코드 생성)에서 한 번에 다 처리하는 게 아니라 단계별로 나뉘서 진행하니까 크게 부담 갖지 않아도 됨.

1. 사용할 툴

툴	용도
Teachable Machine (Pose Project)	카메라로 자세 데이터 촬영 + 학습 + 모델 export
ChatGPT / Gemini	매핑표 정리 + 최종 코드 생성
구글 스프레드시트 / 엑셀	경고 조건 매핑표 정리
메모장(또는 텍스트 편집기)	완성 코드를 .html 파일로 저장
크롬 브라우저	완성된 .html 파일 실행, 웹캠 권한 허용
Framer AI	화면 디자인 시안 참고용 (실제 기능 연결은 안 함)

Figma 안 씬. 화면 기능도 Framer 안에서 안 만들 — 시안만 참고하고 실제 작동은 ChatGPT가 짠 코드로 함. 녹화·재생은 브라우저 기본 기능(MediaRecorder)으로 처리하니까 따로 영상 편집 툴 필요 없음.

2. 전체 구조 한눈에 보기

촬영 가이드 화면

↓

홈 화면 (연습 시작하기 버튼)

↓

카운트다운 (3초)

↓

실시간 녹화 시작

- 웹캠 영상 녹화
- Teachable Machine Pose 모델이 실시간으로 자세 분류
- 같은 자세 지속시간 카운트 → 코칭 문구/화면 반응 표시
- 습관 감지될 때마다 [시간, 클래스] 로그 기록

↓

녹화 종료 (길게 누르기)

↓

분석 중 화면 (로그 데이터 집계)

↓

세션 리포트 화면

- 녹화 영상 재생 (로그 기록된 지점에 마커 표시)
- 구간 타임라인
- 습관 신호 분포 (어떤 습관이 전체의 몇 %)

3. Step 1 — 레퍼런스 조사하기

이번엔 레퍼런스가 이미 정해져 있음. Poise(<https://poise-deploy.vercel.app/>) 접속해서 처음부터 끝까지 한 번 직접 써보기.

체크할 포인트:

- **촬영 가이드 화면:** 카메라를 어떻게 두라고 안내하는지, 이유까지 같이 설명하는지
- **홈 화면:** 인사말 + 직전 세션 요약이 어떻게 배치되는지 (감지 횟수, 지난 회차 대비 변화, 날짜/연습 길이)
- **카운트다운:** 몇 초짜리인지, 어떤 문구가 같이 뜨는지
- **실시간 코칭 문구:** 자세 문제별로 어떤 톤으로 말해주는지 (문제 + 해결 방법을 같이 알려주는 식)

- **분석 중 화면:** 로딩 중에 뭘 보여주는지 (진행률, 안내 문구)
- **세션 리포트:** 재생 플레이어 + 타임라인 + 분포가 한 화면에 어떻게 같이 들어가는지, 다음 연습을 유도하는 문구는 어떤 식으로 쓰는지

캡처는 화면별로 저장해두기. Step 5(Framer 시안) 프롬프트 쓸 때 그대로 참고 자료로 씬.

4. Step 2 — 자세 학습용 영상/사진 촬영하기

Teachable Machine이 자세를 구분할 수 있게, 클래스별 데이터를 카메라로 직접 촬영하는 단계.

4-1. 몇 개나 찍어야 하는지

- 클래스당 **최소 60장(또는 웹캠 녹화 기준 약 20~30초 분량)** 이상 권장. 데이터가 적어도 돌아가긴 하지만, 이 정도는 있어야 실시간으로 썼을 때 오인식이 덜함.
- "정상" 클래스가 제일 많아야 함. 다른 클래스보다 1.5배 정도 넉넉하게 찍기.
- 같은 조명, 같은 각도, 같은 거리에서만 찍으면 실제 연습할 때 (조명이 다르거나 앉는 위치가 조금만 달라져도) 모델이 헛갈려함. 그래서 촬영할 때 **거리(가까울 때/멀 때), 조명(밝을 때/어두울 때), 자세 강도(살짝/확실하게)** 이 세 가지는 최소한 바뀌가면서 찍기.

4-2. Teachable Machine 웹캠으로 바로 촬영하기 (제일 중요한 부분)

사진을 한 장씩 따로 찍어서 업로드하면 시간이 한참 걸림. **Teachable Machine 사이트 안에는 웹캠으로 바로 녹화하는 기능이 있고, 버튼을 누르고 있는 동안 자동으로 여러 프레임을 한꺼번에 캡처함** — 이게 이 프로젝트에서 데이터를 제일 빠르게 모으는 방법이라 꼭 이 방식으로 진행.

1. Teachable Machine 프로젝트 화면에서 클래스 카드 안 **Webcam** 버튼 클릭
2. 웹캠 권한 허용
3. **Hold to Record** 버튼을 누르고 있는 동안 해당 자세를 취하기 → 버튼 누르고 있는 시간만큼 자동으로 여러 장 캡처됨
4. 버튼에서 손 떼면 촬영 끝. 캡처된 프레임들이 클래스 카드 안에 바로 쌓임

별도 촬영 프로그램이나 카메라 앱 필요 없음. 이 사이트 안에서 끝냄.

4-3. 촬영본 정리하기

- Teachable Machine에 웹캠으로 바로 촬영하면 프레임이 각 클래스 카드 안에 자동으로 쌓이기 때문에, **폴더 정리나 파일명 관리는 따로 할 필요 없음**. 이게 전부임 — 클래스 카드 5개 안에 각각 데이터가 들어차 있으면 끝.
- 혹시 폰이나 다른 카메라로 미리 찍어둔 사진/영상을 쓰고 싶으면, 업로드하기 전에 클래스명 그대로 폴더 이름(정상, 시선 회피 등)을 지어서 구분해두는 정도만 하면 됨.

4-4. 클래스 정하기

이 프로젝트는 클래스 5개(정상/시선 회피/고개 움직임/상체 흔들림/발화 멈춤)가 이미 정해져 있어서 그대로 쓰면 됨.

다른 프로젝트로 응용하고 싶다면: 클래스는 **직접 정하는 것부터 시작**임. "어떤 습관을 감지하고 싶은지" 먼저 정하고, 그 습관이 카메라로 구분 가능한 동작인지 확인한 다음 클래스로 만들기. 클래스 개수는 **4~5개 정도가 상한선**. 이보다 많아지면 클래스당 학습 데이터가 쪼개져서 각 클래스의 정확도가 떨어짐. 습관이 더 많다고 느껴져도 비슷한 것끼리 묶어서 5개 안으로 압축하는 게 나음.

5. Step 3 — Teachable Machine에 업로드해서 모델 학습시키기

5-1. Pose Project로 시작하기

1. teachablemachine.withgoogle.com 접속 → **Get Started**
2. **Pose Project** 선택

Image Project 고르면 안 됨. Image Project는 스켈레톤(관절점) 인식이 안 돼서 자세 기반 분류에 못 씬. 실수로 이거 고르는 경우가 많으니까 꼭 확인.

5-2. 클래스 5개 만들고 이름 맞추기

1. 기본으로 Class 1, Class 2 두 개만 있음 → **Add a class** 눌러서 5개까지 늘리기
2. 클래스 이름을 정확히 아래대로 입력

3. 정상
4. 시선 회피
5. 고개 움직임
6. 상체 흔들림
7. 발화 멈춤

오타 나면 안 됨. Step 8(기능 코드)에서 이 이름을 코드가 그대로 문자열로 참조하기 때문에, 띄어쓰기 하나만 달라도 그 클래스를 인식 못 함.

5-3. 각 클래스에 데이터 업로드

4-2에서 설명한 방식대로, 각 클래스 카드의 **Webcam** 버튼 → **Hold to Record**로 클래스 5개 전부 촬영.

5-4. Train Model

1. 가운데 **Training** 패널에서 **Train Model** 버튼 클릭
2. 데이터 양에 따라 다르지만 보통 1~3분 정도면 끝남 (진행바가 다 차면 완료)

5-5. Preview로 확인하고, 안 맞으면 재학습하기

1. 오른쪽 **Preview** 패널에서 웹캠이 자동으로 켜짐
2. 직접 카메라 앞에서 클래스별 자세를 하나씩 취해보면서, 각 클래스 이름 옆에 뜨는 **확률(%)**이 맞게 올라가는지 확인

Train Model이 끝났다고 결과를 그대로 믿으면 안 됨. Preview에서 직접 확인하는 과정을 꼭 거치기. 특정 클래스가 자꾸 다른 걸로 잘못 뜨면, 그 클래스 데이터를 더 추가하고 Train Model을 다시 눌러서 재학습. 한 번에 완벽하게 나오는 경우는 거의 없고, 이 확인-재학습 과정이 정상적인 흐름임.

5-6. Export Model

1. 왼쪽 위 **Export Model** 클릭
2. **Upload (shareable link)** 탭 선택 → **Upload my model** 클릭

3. 업로드 끝나면 `model.json` URL이 뜬 → 이 URL 전체 복사해서 메모장에 따로 저장해두기 (Step 8에서 코드에 넣을 값)

6. Step 4 — 코칭 문구 + 로그 기록 조건표 만들기

Poise는 자세 문제가 감지되면 두 가지를 동시에 함: ① **연습 중 실시간 코칭 문구** ② **나중에 리포트 만들 때 쓸 로그 기록**. 이 표 하나로 둘 다 정리함.

코칭 문구 톤 원칙: Poise는 발표를 "평가"하는 도구가 아니라 혼자 연습하는 사람을 지지적으로 코칭하는 도구를 지향함. 그래서 문구도 "이거 잘못됐어요" 식으로 지적하지 않고, "다음엔 이렇게 해 볼까요?" 처럼 제안하는 톤으로 씀. 아래 6-2에서 AI한테 문구를 만들어달라고 할 때도 이 톤을 꼭 같이 요청하기.

6-1. 클래스별로 몇 단계 경고가 필요한지 정리하기

- "정상"을 뺀 4개 클래스(시선 회피/고개 움직임/상체 흔들림/발화 멈춤) 전부, 지속시간 기준으로 2단계씩 필요함
- 5초 지속 → 1단계 (화면 반응만, 아직 문구는 안 뜸)
- 10초 지속 → 2단계 (코칭 문구 캡션 표시)
- 발화 멈춤만 기준이 다름: 2초 이상 몸이 정지 상태면 바로 감지 (Teachable Machine은 카메라로 웹캠 영상만 보고 소리는 못 들으니까, "무음"이 아니라 "몸이 안 움직이는 상태"로 판단하는 거임 — 말이 멈추면 자연스럽게 몸도 같이 멈추는 경우가 많아서 이 방식으로 대신 감지. 5초까지 기다릴 필요는 없어서 기준을 2초로 짧게 잡음)
- 같은 경고가 반복되지 않도록 전체 공통으로 **8초 쿨다운** 적용

6-2. AI한테 표 초안 만들어달라고 요청하기

위 조건을 ChatGPT나 Gemini한테 주고 표로 정리해달라고 요청.

프롬프트 예시

발표 자세 코칭 앱의 코칭 문구 + 로그 기록 조건표를 만들어줘.

조건:

- 감지 대상 클래스: 시선 회피, 고개 움직임, 상체 흔들림, 발화 멈춤 (정상 제외)
- 시선 회피/고개 움직임/상체 흔들림: 5초 지속 시 1단계(화면 반응만), 10초 지속 시 2단계(코칭 문구 캡션)
- 발화 멈춤: 2초 이상 몸이 정지 상태면 바로 감지, 코칭 문구 캡션 (웹캠은 소리를 못 들으니까 "무음"이 아니라 "몸이 안 움직이는 상태"로 판단)
- 전체 공통으로 같은 경고 반복 시 8초 쿨다운
- 각 클래스·단계별로 실제 화면에 뜰 코칭 문구를 자연스러운 한국어로 만들어줘 (문제 지적하지 말고, "다음엔 이렇게 해볼까요?" 처럼 부드럽게 제안하는 톤 + 해결 방법을 한 문장으로)

표 컬럼: 자세 클래스 | 지속 기준 | 경고 단계 | 코칭 문구 | 화면 반응 | 로그 기록 | 쿨다운
마크다운 표로 출력해줘.

프롬프트 작성 팁: 이 프롬프트에는 4가지가 꼭 들어가야 함

- **지속시간 기준을 숫자로 정확히 주기** — "적당히"가 아니라 "5초, 10초, 2초"처럼 구체적으로 줘야 표에 애매한 값이 안 들어감
- **문구 톤을 지정하기** — "지적하지 말고 제안하는 톤 + 해결 방법을 한 문장으로"처럼 원하는 형식을 말해줘야 Poise 스타일("고개가 기울었어요 — 어깨와 수평을 맞춰볼까요?" 같은 형태)에 가까운 문구가 나옴
- **컬럼 구성을 미리 정해서 주기** — AI가 알아서 표를 짜게 하면 컬럼이 프로젝트마다 제각각으로 나와서, 다음 단계(코드 프롬프트)에 그대로 못 넣게 됨
- **표 형식으로 출력해달라고 명시하기** — 안 그러면 줄글로 풀어서 설명할 때가 있어서 나중에 다시 표로 바꿔달라고 해야 하는 번거로움 생김

(자주 묻는 질문) 엑셀/구글시트로 정리하고 싶은데, 이걸 AI 프롬프트에 어떻게 넣나요? 방법은 두 가지임.

1. **복사-붙여넣기 (무료, 이 방식 추천):** 엑셀이나 구글시트에서 표 영역을 드래그해서 복사(Ctrl+C) → ChatGPT나 Gemini 입력창에 그대로 붙여넣기(Ctrl+V). 표 형태 그대로 인식해서 읽어줌. 2. **파일 첨부 (ChatGPT 유료 버전만 가능):** 입력창 옆 클립 아이콘으로 xlsx 파일 자체를 첨부. 무료 버전은 이 기능이 없어서 대부분 1번 방식으로 진행하면 됨.

이 복사-붙여넣기 방식은 이후 단계(Step 8 코드 생성 프롬프트에 매핑표 붙여넣을 때)에서도 똑같이 쓰면 됨.

6-3. 결과 검토하기

AI가 만들어준 표를 그대로 믿지 말고, 최소 2개 클래스 분량은 직접 눈으로 대조 검산하기. 아래는 검산 기준으로 삼을 수 있는 직접 작성 예시.

자세 클래스	지속 기준	경고 단계	코칭 문구	화면 반응	로그 기록	쿨다운
시선 회피	5초	1단계	(없음, 화면 반응만)	화면 테두리 색 변경	0	-
시선 회피	10초	2단계	"시선을 카메라 쪽으로 옮겨볼까요?"	코칭 문구 캡션 표시	0	8초
발화 멈춤	2초 (정지)	1단계	"잠시 멈췄네요, 천천히 다시 시작해볼까요?"	코칭 문구 캡션 표시	0	8초

AI가 준 표랑 이 예시를 비교해서, 컬럼 이름이나 기준값이 이상하게 바뀐 게 없는지 확인. 특히 지속시간 숫자가 원래 조건(5초/10초/2초)이랑 다르게 나오는 경우가 종종 있어서 이 부분 꼭 대조.

7. Step 5 — Framer AI로 화면 시안 만들기

Poise 화면 6개 중 핵심 5개(카운트다운 제외, 나머지는 간단해서 스킵 가능)를 화면별로 하나씩 만들기. 한 번에 다 시키지 말고 나눠서 요청.

① 촬영 가이드 화면 프롬프트

발표 코칭 앱의 첫 화면을 만들어줘.

- 카메라를 어떻게 두면 좋은지 안내 문구
- 카메라 배치 예시 아이콘/일러스트
- 하단에 "시작하기" 버튼
- 세로형 모바일 레이아웃 (375x812)

② 홈 화면 프롬프트

발표 코칭 앱의 홈 화면을 만들어줘.

- 상단 인사말
- "오늘도 발표 연습해볼까요?" 문구
- 직전 세션 요약 카드 (감지 횟수, 지난 회차 대비 변화 화살표, 날짜/연습 시간)
- "연습 시작하기" 큰 버튼
- 세로형 모바일 레이아웃, 다크 배경 + 포인트 컬러

③ 실시간 녹화 화면 프롬프트

발표 코칭 앱의 실시간 녹화 화면을 만들어줘.

- 전체 화면을 채우는 웹캠 영상 영역
- 상단에 경과 시간 타이머
- 하단에 실시간 코칭 문구 캡션 박스
- 경고 단계에 따라 화면 테두리 색이 바뀌는 상태
- "길게 눌러 종료" 안내 문구

④ 분석 중 화면 프롬프트

발표 코칭 앱의 분석 중 로딩 화면을 만들어줘.

- 진행률 % 표시
- "습관 신호를 집계하는 중이에요" 문구

⑤ 세션 리포트 화면 프롬프트

발표 코칭 앱의 세션 리포트 화면을 만들어줘.

- 상단에 녹화 영상 재생 플레이어 (재생바 포함)
- 재생바 아래 구간 타임라인 (습관 감지된 지점에 색 마커)
- 습관 신호 분포 (막대 또는 도넛 차트, 클래스별 비율)
- 하단에 "다시 연습" 버튼

프롬프트 작성 팁: 이 프롬프트에는 4가지가 꼭 들어가야 함

- **화면 하나씩 나눠서 요청하기** — 5개를 한 번에 시키면 화면끼리 스타일이 뒤섞여서 나올 때가 많음
- **필요한 요소를 빠짐없이 나열하기** — 리스트로 꼭 적어줘야 AI가 빠뜨리지 않음
- **레이아웃 크기를 명시하기** — "세로형 모바일 375x812"처럼 숫자로 줘야 화면 비율이 일정하게 나옴
- **톤/컬러를 지정하기** — 안 주면 AI가 임의로 밝은 톤이나 화려한 그라데이션을 넣을 때가 있음. 다크+포인트 컬러 톤은 자유롭게 정해도 됨

요소마다 영문 이름 붙이기

화면 시안이 완성되면, 나중에 코드에서 각 요소를 정확히 가리킬 수 있도록 **영문 이름표**를 붙여야 함.
Framer 왼쪽 **Layers** 탭에서 요소 클릭 → 이름 더블클릭 → 아래 규칙대로 이름 바꾸기.

이름 짓는 규칙: 역할을 알 수 있는 영문 단어 + 역할 접미사(Button, Badge, Banner, Timer, View 등)

화면	요소	이름
촬영 가이드	시작하기 버튼	StartGuideButton
홈	연습 시작하기 버튼	StartSessionButton
홈	직전 세션 요약 카드	LastSessionCard
실시간 녹화	웹캠 화면 영역	WebcamView
실시간 녹화	경과 시간 타이머	PresentationTimer
실시간 녹화	코칭 문구 캡션 박스	CoachingCaption
세션 리포트	영상 재생 플레이어	ReplayPlayer
세션 리포트	구간 타임라인	HabitTimeline
세션 리포트	습관 분포 차트	HabitDistributionChart
세션 리포트	다시 연습 버튼	RetryButton

이 이름표를 그대로 다음 단계(Step 6) 프롬프트에 넣어서 id로 쓸 거임.

8. Step 6 — ChatGPT로 완성 코드 받기

이 단계가 제일 헷갈릴 수 있는 부분이라, "어디서 뭘 누르고 뭘 붙여넣는지"를 클릭 단위로 쪼갬. 딱 순서대로만 따라가면 됨.

전체 흐름은 이거임: **같은 채팅창 안에서** ① 화면(디자인)만 먼저 받기 → ② 그 화면에 기능(로직)을 이어서 붙여받기. 두 번에 나눠서 시키는 이유는, 한 번에 다 시키면 AI가 버튼 동작을 마음대로 추측해서 넣어버리거나 코드가 뒤섞여서 나중에 고치기 어려워지기 때문.

꼭 같은 채팅창 안에서 이어가기. 8-2를 새 채팅으로 시작하면 ChatGPT가 8-1에서 만든 화면 코드를 기억 못 해서 처음부터 다시 설명해야 함.

8-1. 디자인 시안 → HTML/CSS 변환 (기능 없이 화면만)

8-1-1. ChatGPT 새 채팅 열기

1. 크롬에서 chatgpt.com 접속
2. 로그인 (무료 계정으로 충분함)
3. 왼쪽 위 **새 채팅** 버튼 클릭

8-1-2. 시안 이미지 5장 첨부하기

1. 화면 아래 입력창 왼쪽에 있는 **+ 아이콘**(또는 클립 아이콘) 클릭
2. **사진 및 파일 추가** 선택
3. Step 5에서 캡처해둔 시안 이미지 5장을 **순서대로**(촬영가이드→홈→실시간녹화→분석중→세션리포트) 선택해서 업로드
4. 입력창 위에 이미지 5장 썸네일이 뜨면 업로드 완료

8-1-3. 프롬프트 입력하고 전송하기

아래 프롬프트를 통째로 복사(Ctrl+C)해서 입력창에 붙여넣기(Ctrl+V) → 전송(Enter 또는 종이비행기 아이콘)

프롬프트 예시

첨부한 이미지들은 발표 자세 코칭 앱의 화면 5개 시안이야
(촬영가이드/홈/실시간녹화/분석중/세션리포트, 이 순서대로 첨부함).

이 디자인을 참고해서 HTML/CSS로 정적인 화면(기능 없이 레이아웃과 스타일만)을 만들어줘.

- 기능은 넣지 마. 지금은 화면만 만드는 단계야.
- 화면 5개를 각각 다른 div 섹션으로 나눠주고, id는 아래 이름표 그대로 써줘.
 - 촬영가이드: id="guide", 안의 버튼은 id="StartGuideButton"

```
- 홈: id="home", 안의 카드는 id="LastSessionCard", 버튼은 id="StartSessionButton"
- 실시간녹화: id="recording", 웹캠 영역은 id="WebcamView", 타이머는
id="PresentationTimer", 캡션은 id="CoachingCaption"
- 분석중: id="analyzing"
- 세션리포트: id="report", 재생 플레이어는 id="ReplayPlayer", 타임라인은
id="HabitTimeline", 분포차트는 id="HabitDistributionChart", 버튼은
id="RetryButton"
- 처음엔 촬영가이드 화면만 보이게 해줘 (나머지는 display:none)
- 세로형 모바일 레이아웃 기준으로 짜줘 (최대 너비 480px 정도, 화면 중앙 정렬)
- 웹캠 넣을 자리는 지금은 회색 박스로
- 색상은 다크 배경(#17171C) + 포인트 컬러(#5566FF) 톤으로 CSS 짜줘 (이미지랑 색이 조금 다
를 수 있어서 헥스코드 다시 한번 정확히 줌)
- 완성된 코드는 html, css, javascript 구분 없이 파일 하나(index.html) 안에 전부 합쳐서
줘
```

프롬프트 작성 팁: 이 프롬프트에는 5가지가 꼭 들어가야 함

- **화면 순서와 개수를 먼저 알려주기** — 이미지가 몇 장인지, 어떤 순서인지 말해줘야 AI가 각 이미지를 헷갈리지 않고 매칭함
- **"기능은 넣지 마"를 명시하기** — 안 그러면 AI가 알아서 버튼에 동작을 추측해서 넣어버려서, 다음 단계(8-2)에서 오히려 코드가 꼬임
- **이름표를 id로 그대로 지정하기** — Step 5에서 정리한 이름표를 여기서 바로 넣어줘야, 8-2에서 그 id를 정확히 가리켜서 기능을 붙일 수 있음
- **색상 값을 다시 한번 정확히 주기** — 이미지에서 색이 조금 다르게 나올 수 있어서, 헥스코드를 프롬프트에 또 적어주는 게 안전함
- **화면을 어떻게 쪼갤지 말해주기** — "화면 5개를 각각 다른 div 섹션으로 나눠줘"라고 요청하면 화면마다 상자가 따로따로 만들어짐. 이렇게 미리 나눠놔야 8-2에서 "지금은 홈 화면 상자만 보여주고 나머지는 숨겨줘" 같은 코드를 붙이기 쉬움

8-1-4. 받은 코드 확인하고 그대로 두기

1. ChatGPT가 답변을 주면, 회색 박스(코드블록) 안에 코드가 쭉 나옴
2. 지금은 따로 저장하지 않아도 됨 — 같은 채팅창에서 바로 이어서 8-2를 진행할 거라, ChatGPT가 이 코드를 계속 기억하고 있음
3. 코드가 중간에 끊겨서 나오면(예: 태그가 안 닫힌 채로 끝남) 입력창에 이어서 계속 작성해줘라고 보내면 나머지를 이어서 줌

8-2. 실제 작동하는 기능 코드 작성 (데이터 연동 + 로직)

같은 채팅창에서 이어서 진행. 8-1에서 만든 화면에 진짜 웹캠 인식, 녹화, 분석, 리포트 기능을 붙이는 단계.

8-2-1. 매핑표를 프롬프트에 붙여넣을 준비하기

1. Step 4에서 정리한 표(구글 스프레드시트 또는 엑셀) 열기
2. 표 전체를 마우스로 드래그해서 선택 → 복사(Ctrl+C)

8-2-2. 프롬프트 작성하고 전송하기

아래 프롬프트를 통째로 복사해서 붙여넣은 다음, [Step 4 표 붙여넣기] 라고 써있는 부분을 지우고 그 자리에 방금 복사한 표를 붙여넣기(Ctrl+V). 그다음 전송.

프롬프트 예시

아래는 발표 자세 코칭 앱의 코칭 문구 + 로그 기록 조건표야.

[Step 4 표 붙여넣기]

이 데이터를 가지고 위에서 만든 화면 5개(guide/home/recording/analyzing/report)에 실제 기능을 연결한 완성 코드를 짜줘. 조건은 다음과 같아.

- html, css, javascript를 전부 파일 하나(index.html)에 합쳐줘. 더블클릭 없이 웹캠 권한만 허용하면 바로 실행되게.
- Teachable Machine의 tmPose 라이브러리를 CDN 스크립트 태그로 불러와줘.
- 모델 URL은 아래 변수에 넣을 거니까 그 자리를 명확하게 표시해줘.
`const MODEL_URL = "여기에_복사한_URL_붙여넣기";`
- 화면 전환은 id 기준으로: StartGuideButton 클릭 → home 보여주기 → StartSessionButton 클릭 → 카운트다운(3초) → recording 보여주기 → 길게 누르면 종료 → analyzing → report
- WebcamView 안에 실제 웹캠 영상을 넣어줘 (웹캠 접근 권한 요청 처리 포함, 거부됐을 때 안내 문구도 띄워줘)
- 예측 결과가 85% 이상일 때만 해당 자세로 판단하도록 해줘 (그 이하는 무시).
- 같은 자세가 계속 감지될 때 지속시간(초)을 카운트하는 로직을 넣어줘. 다른 자세로 바뀌면 카운트 리셋.
- 표 기준으로 지속시간 도달하면 CoachingCaption에 해당 문구를 띄워줘.
- 같은 경고가 8초 안에 다시 뜨지 않도록 쿨다운 로직을 넣어줘.
- 자세가 감지될 때마다 {시간(초), 클래스} 형태로 로그 배열에 저장해줘. 이게 나중에 리포트용 데이터야.
- 녹화는 MediaRecorder API로 WebcamView 영상을 녹화해서, 종료 시점에 Blob URL로 만들어줘.
- analyzing 화면은 2~3초 정도 가짜 로딩(진행률 % 애니메이션)만 보여주고 report로 넘어가게 해줘.

- report 화면에서: 녹화한 영상을 ReplayPlayer에 재생, 로그 배열을 집계해서 클래스별 비율(%)을 HabitDistributionChart에 표시, 로그 시간을 기준으로 HabitTimeline에 마커 표시.
- 세션이 끝나면 이번 세션 요약(감지 횟수, 날짜, 연습 시간)을 localStorage에 저장해서, 다음에 홈 화면 열 때 LastSessionCard에 보여줘.
- 한글 주석 달아줘. 초보자도 읽을 수 있게.

프롬프트 작성 팁: 이 프롬프트에는 6가지가 꼭 들어가야 함

- **매핑표를 통째로 붙여넣기** — 8-2-1에서 복사해둔 표를 그대로 붙여넣기. 이게 6-2 FAQ에서 설명한 복사-붙여넣기 방식임
- **파일을 합쳐달라고 명시하기** — "index.html 하나로, 외부 파일 연결 없이"라고 안 하면 css/js를 별도 파일로 분리해서 줄 때가 있어서 메모장 저장 단계에서 곤란해짐
- **요소 id 이름으로 콧 집어서 지시하기** — "버튼 누르면"처럼 두루뭉술하게 말하지 말고, Step 5에서 붙인 이름을 그대로 써서 "StartSessionButton을 클릭하면"처럼 구체적으로 지시. 그래야 원하는 요소에 원하는 기능이 정확히 연결됨
- **모델 URL 변수 자리를 명확히 지정하기** — `const MODEL_URL = "..."` 형태로 정확히 요청해야 나중에 값만 바꿔치기하기 쉬움
- **판단 기준값(85%, 8초 등)을 숫자로 정확히 주기** — 애매하게 두면 AI가 임의 값을 넣어버림
- **주석 요청하기** — 코드를 이해하고 넘어가려면 한글 주석을 꼭 요청

8-2-3. 완성 코드 받고 복사하기

1. ChatGPT가 코드블록으로 완성 코드를 줌
2. 코드가 길어서 중간에 멈추면(태그가 안 닫힌 채 끝나거나 "... 로 생략된 부분이 있으면) 이어서 계속 작성해줘라고 보내서 나머지 받기 — 이번엔 화면 5개 + 녹화/재생 로직까지 있어서 코드가 길고, 한 번에 안 끝나는 게 오히려 정상임
3. 코드블록 오른쪽 위(또는 위쪽)에 있는 **복사 아이콘** 클릭 → 코드 전체가 클립보드에 복사됨 (아이콘이 안 보이면 코드블록 안을 클릭 → 전체 선택(Ctrl+A) → 복사(Ctrl+C))
4. 다음 단계(Step 7)로 넘어가서 이 코드를 메모장에 붙여넣기

모델 URL 자리는 Step 3-6에서 복사해둔 `model.json` 링크로 실제 코드에서 바꿔치기해야 함.
ChatGPT가 준 코드 그대로 두면 작동 안 함 — 이 작업은 Step 7에서 함.

9. Step 7 — 메모장에 저장해서 브라우저로 열기

1. ChatGPT가 준 코드 전체를 복사 (``html 코드블록 안의 내용 전부)
 2. **메모장** 실행 (윈도우 시작 메뉴에서 "메모장" 검색)
 3. 복사한 코드를 메모장에 붙여넣기
 4. `const MODEL_URL = "여기에_복사한_URL_붙여넣기";` 부분 찾아서, Step 3-6에서 저장해둔 실제 `model.json` URL로 바꾸기
 5. 메모장 상단 **파일 → 다른 이름으로 저장**
 6. 저장 창에서
 7. 파일 이름: `posture_coach.html` (뒤에 `.html`까지 직접 입력)
 8. 파일 형식: **모든 파일**로 변경
 9. 저장한 파일 **더블클릭**
 10. 메모장으로 열리면: 파일 우클릭 → 연결 프로그램 → **Chrome** 선택해서 다시 열기
 11. 크롬이 웹캠 권한 요청 팝업을 띄우면 반드시 **허용** 누르기 (로컬 파일로 열어도 대부분 정상 작동함)
 12. 촬영가이드 화면부터 순서대로 넘어가면서 웹캠 영상이 뜨고 자세 인식이 되면 성공
- 그래도 웹캠이 안 켜지면 11번 트러블슈팅에서 확인.

10. Step 8 — 테스트하기

전체 흐름(촬영가이드 → 홈 → 카운트다운 → 실시간녹화 → 분석중 → 세션리포트)을 처음부터 끝까지 한번 통으로 실행.

- 실시간 녹화 화면에서 클래스 3개(시선 회피/고개 움직임/상체 흔들림)를 직접 자세로 취해보면서 5초/10초 시점에 코칭 문구가 뜨는지 확인
- "발화 멈춤"은 몸을 2초 이상 움직이지 않고 가만히 있어서 캡션이 뜨는지 확인
- 같은 경고 뜬 직후 바로 다시 취했을 때 8초 안에는 안 뜨는지 (쿨다운) 확인
- 길게 눌러서 녹화 종료했을 때 분석중 화면으로 잘 넘어가는지 확인

- 세션리포트에서 녹화 영상이 재생되는지, 타임라인 마커가 실제 감지 시점이랑 맞는지, 습관 분포 %가 대충 맞는지 확인
- 세션리포트 확인 후 홈 화면으로 돌아갔을 때, LastSessionCard에 방금 한 연습 정보가 뜨는지 확인 (localStorage 저장 확인)

11. 자주 나는 문제

증상	원인	해결
웹캠 권한 팝업이 안 뜨거나 거부됨	이전에 권한을 거부한 상태로 남아 있음	크롬 주소창 왼쪽 자물쇠(또는 정보) 아이콘 클릭 → 카메라 권한을 "허용"으로 변경 후 새로고침
화면이 계속 로딩만 되고 안 뜸	model.json URL이 잘못 들어감	코드에서 MODEL_URL 값이 Step 3-6에서 복사한 링크랑 정확히 같은지 확인 (오타, 공백 확인)
자세를 자주 잘못 인식함	학습 데이터가 부족하거나 각도가 안 다양함	Teachable Machine으로 돌아가서 헛갈리는 클래스 데이터 추가 촬영 후 재학습 → Export Model 다시
코칭 문구가 너무 자주 뜸	쿨다운 로직이 코드에 제대로 안 들어감	F12 → Console에서 에러 확인, ChatGPT한테 "쿨다운 로직이 안 걸리는 것 같다"고 코드 붙여넣고 다시 요청
녹화 종료했는데 세션리포트에서 영상이 안 나옴	MediaRecorder Blob URL 처리가 빠졌거나 코드가 중간에 끊김	ChatGPT한테 "MediaRecorder로 녹화한 영상을 ReplayPlayer에 재생하는 부분만 다시 짜줘"로 재요청
홈 화면에 직전 세션 요약이 안 뜸	localStorage 저장/불러오기 코드가 빠짐	F12 → Console에서 localStorage 입력해서 데이터 저장됐는지 확인, 안 됐으면 저장 로직 다시 요청
한글이 이상하게 깨져 보임	저장할 때 인코딩이 맞지 않음	메모장 저장 창에서 인코딩 옵션을 UTF-8로 다시 선택하고 저장
소리가 안 남	브라우저 자동재생 정책 때문에 사용자 클릭 전에는 오디오 재생이 막혀있음	코칭 문구에 소리를 추가하고 싶다면, 화면에 버튼을 하나 넣어서 그 버튼을 누른 이후부터 오디오가 재생되게 함

12. 마무리 체크리스트

- [] Step 1: Poise 앱 직접 써보고 화면 흐름 6단계 캡처해줬다
- [] Step 2 (4-1~4-4): 클래스 5개 각각 최소 60장(또는 20~30초 분량) 이상 촬영했다, 거리·조명·강도 다양하게 찍었다
- [] Step 3: Teachable Machine **Pose Project**로 학습했다 (Image Project 아님), Preview 확인 후 필요하면 재학습했다
- [] Step 3: Export Model로 model.json URL 받아서 저장해줬다
- [] Step 4 (6-1~6-3): 코칭 문구 + 로그 기록 조건표 다 채우고, 표본 2개 클래스는 직접 대조 검사했다
- [] Step 5: 화면 시안 5개(촬영가이드/홈/실시간녹화/분석중/세션리포트) 만들고 요소마다 영문 이름표 붙였다
- [] Step 6 (8-1): 화면만 있는 정적 HTML/CSS 받았다 (id 이름표 반영됨)
- [] Step 6 (8-2): 완성 코드(index.html) 받고 MODEL_URL 실제 값으로 바꿨다
- [] Step 7: 메모장 → .html 저장(모든 파일) → 크롬 실행 → 웹캠 권한 허용까지 확인했다
- [] Step 8: 화면 흐름 전체(촬영가이드→홈→카운트다운→녹화→분석중→리포트) 순서대로 다 테스트 해봤다
- [] Step 8: 세션리포트에서 녹화 재생 + 타임라인 + 습관 분포까지 확인했다

여기까지 되면 배포 없이도 프로토타입 완성. 데모 발표할 땐 이 html 파일 하나만 노트북에 들고 가서 크롬으로 열면 됨.